Course Code: DSA03	Course Name: Natural Language Processing
Branch: CSE	Year: 2026
Name: Prajiith K	Reg No: 192324013
Date: 05/11/2026	Dept: AI&DS

Assignment 3

Question 1

Draw tree structures for the following ATIS sentences: (10M) (CO3) (BLT4)

a. Does American airlines have a flight between five a.m. and six a.m.

b. I would like to fly on American airlines.

c. Please repeat that.

d. Does American 487 have a first class section?

e. I need to fly between Philadelphia and Atlanta.

f. What is the fare from Atlanta to Denver?

g. Is there an American airlines flight from Philadelphia to Dallas?

Approach

Each sentence is parsed with a context-free grammar (CFG) written for the ATIS domain. The grammar is loaded into the NLTK chart parser, which returns the parse tree of every sentence. The tree is printed in indented bracket notation and is then drawn as a tree diagram by a small drawing module. The grammar is written so that each of the seven sentences has exactly one parse. The full stop and question mark at the end of a sentence are not part of the grammar, whereas the full stops inside a.m. belong to the word.

Grammar used

Syntactic rules	Lexical rules
S -> Aux NP VP S -> NP VP S -> Intj VP S -> Verb Expl NP S -> WhNP Verb NP WhNP -> Wh NP -> Pronoun NP -> PropN NP -> PropN Noun NP -> PropN Num NP -> Num Noun NP -> Det Nominal NP -> Det Nominal PP NP -> Det Nominal PP PP Nominal -> Noun	Aux -> does Modal -> would Verb -> have | like | fly repeat | need | is Pronoun -> I | that PropN -> American | Philadelphia Atlanta | Denver | Dallas Noun -> airlines | flight | fare class | section | a.m. Num -> five | six | 487 Det -> a | an | the Adj -> first Prep -> between | on | from | to To -> to Conj -> and

```
Nominal -> Adj Nominal
Nominal -> PropN Nominal
Nominal -> Noun Nominal
VP      -> Verb NP
VP      -> Verb PP
VP      -> Verb InfVP
VP      -> Modal VP
InfVP   -> To VP
PP      -> Prep NP
PP      -> Prep NP Conj NP
```

```
Intj    -> Please
Expl    -> there
Wh      -> What
```

Program: tree_draw.py (helper module for drawing trees)

```
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt

FONT_SIZE = 13
CHAR_WIDTH = 0.115
GAP = 0.4
ROW_HEIGHT = 0.62



def is_preterminal(node):
    if isinstance(node, str):
        return False
    return len(node) == 1 and isinstance(node[0], str)



def slot_width(node):
    if isinstance(node, str):
        return len(node) * CHAR_WIDTH + GAP
    if is_preterminal(node):
        return max(len(node.label()), len(node[0])) * CHAR_WIDTH + GAP
    return sum(slot_width(child) for child in node)



def layout(node, depth, left, nodes, edges):
    width = slot_width(node)
    index = len(nodes)
    if is_preterminal(node):
        centre = left + width / 2
        nodes.append([centre, depth, node.label(), False])
        nodes.append([centre, depth + 1, node[0], True])
        edges.append((index, index + 1))
        return index
    nodes.append([0.0, depth, node.label(), False])
    children = []
    cursor = left
    for child in node:
        children.append(layout(child, depth + 1, cursor, nodes, edges))
        cursor += slot_width(child)
    nodes[index][0] = sum(nodes[c][0] for c in children) / len(children)
    for child in children:
        edges.append((index, child))
    return index



def save_tree(tree, path):
    nodes = []
    edges = []
    layout(tree, 0, 0.0, nodes, edges)
    bottom = max(n[1] for n in nodes if not n[3])
    for n in nodes:
```

```
        if n[3]:
            n[1] = bottom + 1
    total = slot_width(tree)
    figure, axis = plt.subplots(figsize=(total, (bottom + 2) * ROW_HEIGHT))
    axis.set_xlim(0, total)
    axis.set_ylim(-(bottom + 1.6), 0.6)
    axis.axis("off")
    for parent, child in edges:
        px, pd = nodes[parent][0], nodes[parent][1]
        cx, cd = nodes[child][0], nodes[child][1]
        axis.plot(
            [px, cx], [-pd - 0.2, -cd + 0.22], color="black", linewidth=1.0
        )
    for x, depth, label, leaf in nodes:
        axis.text(
            x,
            -depth,
            label,
            ha="center",
            va="center",
            fontsize=FONT_SIZE,
            family="DejaVu Serif",
            style="normal",
            fontweight="normal",
            color="#7a1f1f" if leaf else "black",
        )
    figure.savefig(
        path, dpi=200, bbox_inches="tight", pad_inches=0.1, facecolor="white"
    )
    plt.close(figure)
```

Program: q1_atis_trees.py

```
import nltk

from tree_draw import save_tree

GRAMMAR = nltk.CFG.fromstring(
    """
    S -> Aux NP VP
    S -> NP VP
    S -> Intj VP
    S -> Verb Expl NP
    S -> WhNP Verb NP
    WhNP -> Wh
    NP -> Pronoun
    NP -> PropN
    NP -> PropN Noun
    NP -> PropN Num
    NP -> Num Noun
    NP -> Det Nominal
    NP -> Det Nominal PP
    NP -> Det Nominal PP PP
    Nominal -> Noun
    Nominal -> Adj Nominal
    Nominal -> PropN Nominal
    Nominal -> Noun Nominal
    VP -> Verb NP
    VP -> Verb PP
    VP -> Verb InfVP
    VP -> Modal VP
    InfVP -> To VP
    PP -> Prep NP
    PP -> Prep NP Conj NP
    Aux -> 'does'
    Modal -> 'would'
    Verb -> 'have' | 'like' | 'fly' | 'repeat' | 'need' | 'is'
    Pronoun -> 'i' | 'that'
    PropN -> 'american' | 'philadelphia' | 'atlanta' | 'denver' | 'dallas'
    Noun -> 'airlines' | 'flight' | 'fare' | 'class' | 'section' | 'a.m.'
```

```
    Num -> 'five' | 'six' | '487'
    Det -> 'a' | 'an' | 'the'
    Adj -> 'first'
    Prep -> 'between' | 'on' | 'from' | 'to'
    To -> 'to'
    Conj -> 'and'
    Intj -> 'please'
    Expl -> 'there'
    Wh -> 'what'
    """
)

SENTENCES = {
    "a": "Does American airlines have a flight between five a.m. and six a.m.",
    "b": "I would like to fly on American airlines",
    "c": "Please repeat that",
    "d": "Does American 487 have a first class section",
    "e": "I need to fly between Philadelphia and Atlanta",
    "f": "What is the fare from Atlanta to Denver",
    "g": "Is there an American airlines flight from Philadelphia to Dallas",
}



def restore_case(tree, words):
    for position, word in zip(tree.treepositions("leaves"), words):
        tree[position] = word
    return tree



def main():
    parser = nltk.ChartParser(GRAMMAR)
    for key, sentence in SENTENCES.items():
        words = sentence.split()
        parses = list(parser.parse([word.lower() for word in words]))
        tree = restore_case(parses[0].copy(deep=True), words)
        print(f"Sentence ({key}): {sentence}")
        print(f"Number of parses: {len(parses)}")
        print(tree.pformat(margin=60))
        print()
        save_tree(tree, f"tree_{key}.png")



if __name__ == "__main__":
    main()
```

Output and tree structures

a. Does American airlines have a flight between five a.m. and six a.m.

The sentence is a yes/no question, so the rule S -> Aux NP VP is used. The auxiliary Does opens the question, the subject NP American airlines is a PropN followed by a Noun, and the VP is the Verb have followed by an object NP. The object NP is built as Det Nominal PP: a flight, modified by the

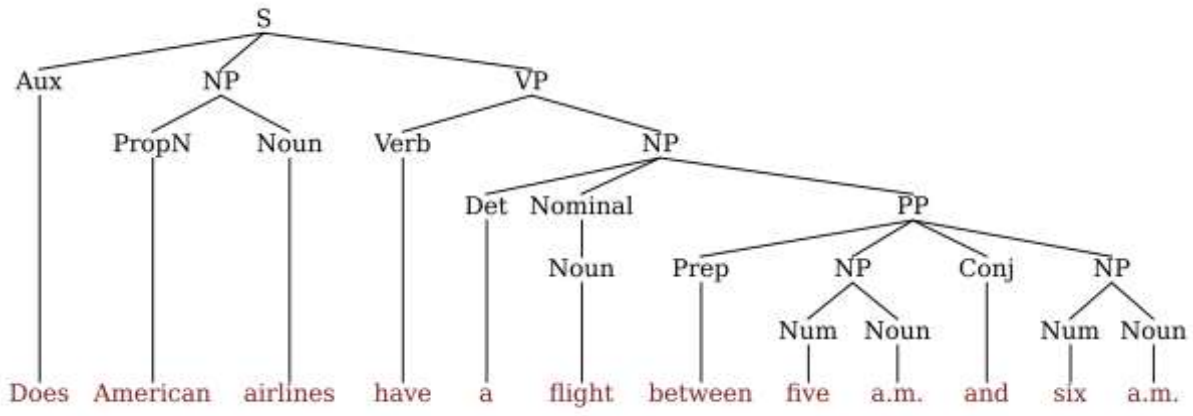
prepositional phrase between five a.m. and six a.m., which follows the pattern Prep NP Conj NP with each time expression forming an NP of Num and Noun.

Output:

```
python q1_atis_trees.py

Sentence (a): Does American airlines have a flight between five a.m. and six a.m.
Number of parses: 1
(S
  (Aux Does)
  (NP (PropN American) (Noun airlines))
  (VP
    (Verb have)
    (NP
      (Det a)
      (Nominal (Noun flight))
      (PP
        (Prep between)
        (NP (Num five) (Noun a.m.))
        (Conj and)
        (NP (Num six) (Noun a.m.))))))
```

Tree structure:



b. I would like to fly on American airlines.

This is a declarative sentence with the rule S -> NP VP, where the subject NP is the Pronoun I. The VP is nested: the Modal would takes a VP headed by the Verb like, which takes the infinitive phrase

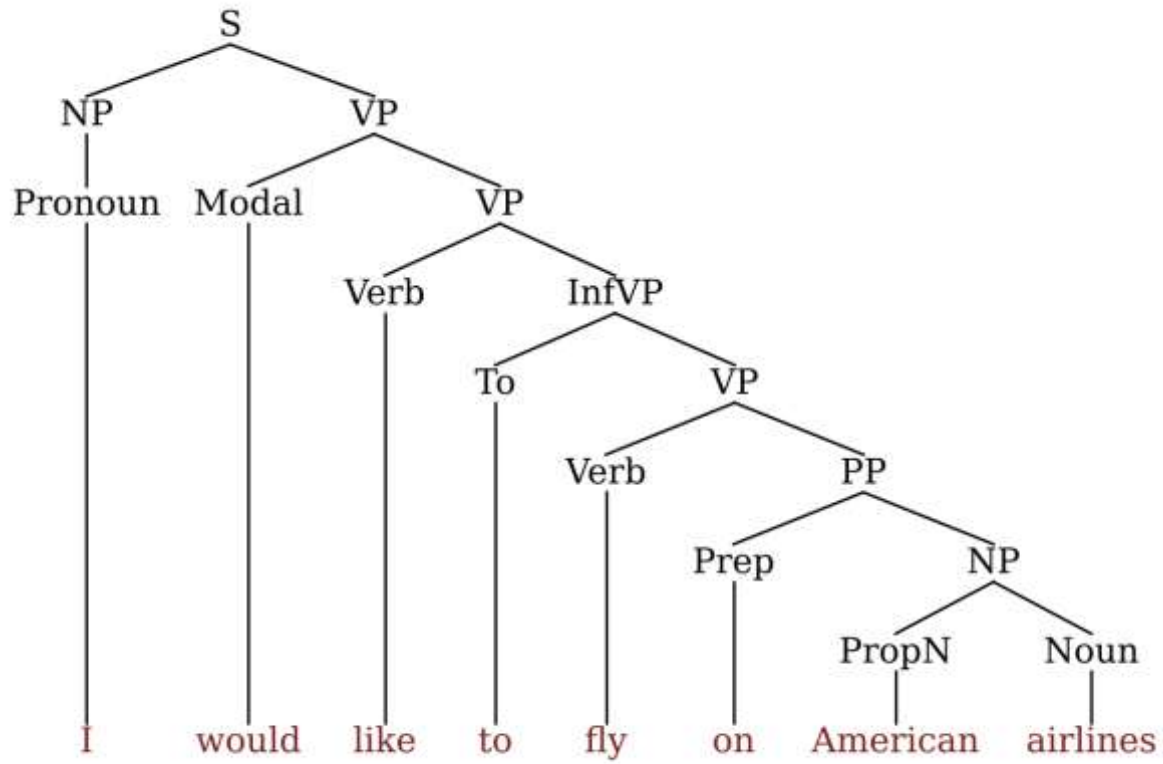
InfVP -> To VP. The innermost VP is the Verb fly followed by the prepositional phrase on American airlines.

Output:

```
python q1_atis_trees.py

Sentence (b): I would like to fly on American airlines
Number of parses: 1
(S
  (NP (Pronoun I))
  (VP
    (Modal would)
    (VP
      (Verb like)
      (InfVP
        (To to)
        (VP
          (Verb fly)
          (PP
            (Prep on)
            (NP (PropN American) (Noun airlines))))))))
```

Tree structure:



c. Please repeat that.

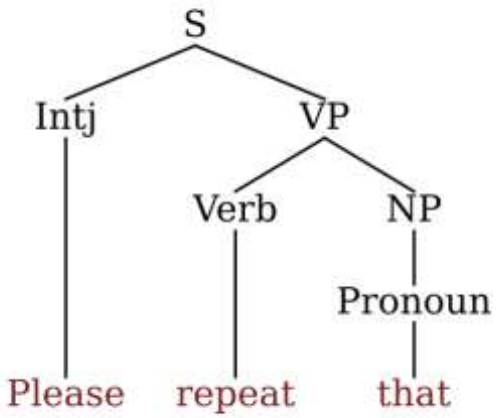
This is an imperative sentence with no subject, so the rule S -> Intj VP is used. The interjection Please is followed by a VP consisting of the Verb repeat and an object NP formed by the Pronoun that.

Output:

```
python q1_atis_trees.py

Sentence (c): Please repeat that
Number of parses: 1
(S (Intj Please) (VP (Verb repeat) (NP (Pronoun that))))
```

Tree structure:



d. Does American 487 have a first class section?

The yes/no question pattern S -> Aux NP VP is used again. The subject NP is a PropN followed by a Num, because American 487 is an airline name with a flight number. The object NP is Det Nominal,

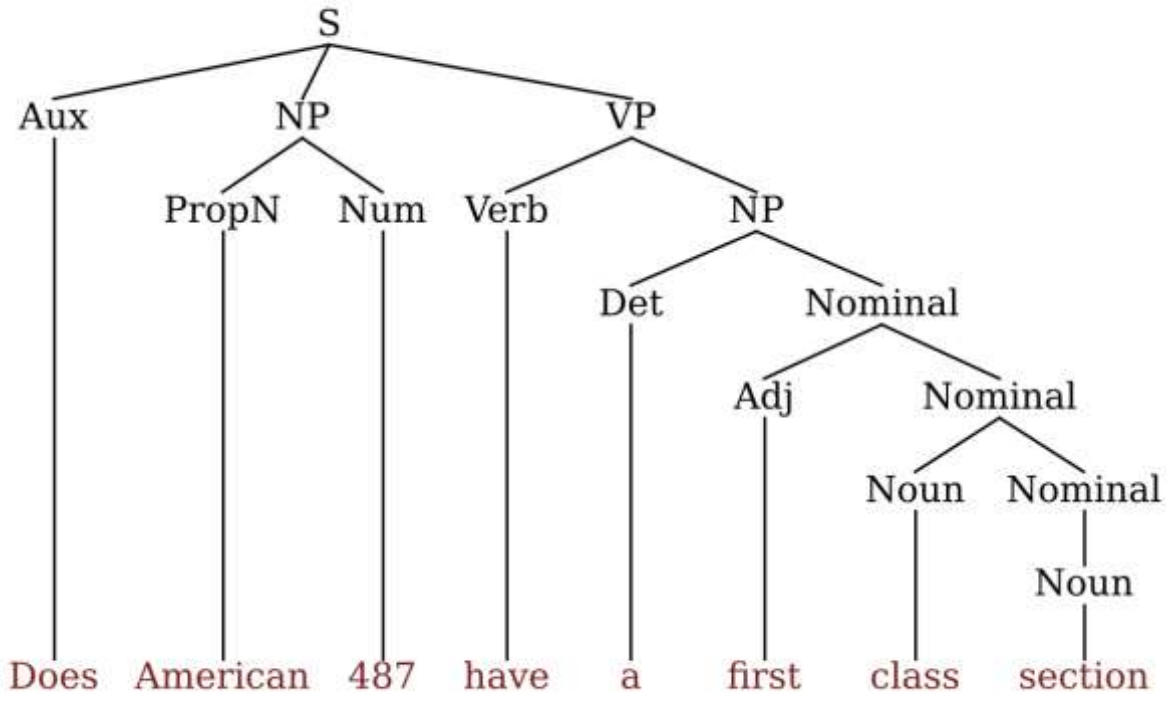
where the Nominal is nested: the adjective first modifies the compound class section, built from two Nouns.

Output:

```
python q1_atis_trees.py

Sentence (d): Does American 487 have a first class section
Number of parses: 1
(S
  (Aux Does)
  (NP (PropN American) (Num 487))
  (VP
    (Verb have)
    (NP
      (Det a)
      (Nominal
        (Adj first)
        (Nominal (Noun class) (Nominal (Noun section)))))))
```

Tree structure:



e. I need to fly between Philadelphia and Atlanta.

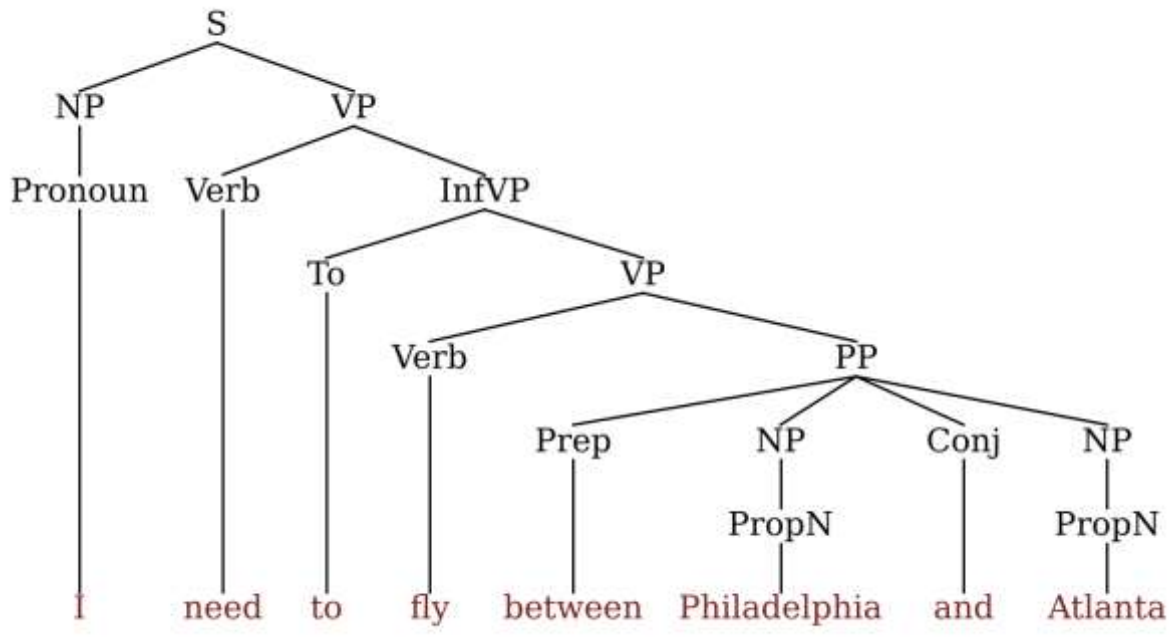
The sentence follows S -> NP VP with the Pronoun I as the subject. The VP is the Verb need followed by the infinitive phrase to fly, and the inner VP is the Verb fly followed by the prepositional phrase between Philadelphia and Atlanta, which uses PP -> Prep NP Conj NP.

Output:

```
python q1_atis_trees.py

Sentence (e): I need to fly between Philadelphia and Atlanta
Number of parses: 1
(S
  (NP (Pronoun I))
  (VP
    (Verb need)
    (InfVP
      (To to)
      (VP
        (Verb fly)
        (PP
          (Prep between)
          (NP (PropN Philadelphia))
          (Conj and)
          (NP (PropN Atlanta)))))))
```

Tree structure:



f. What is the fare from Atlanta to Denver?

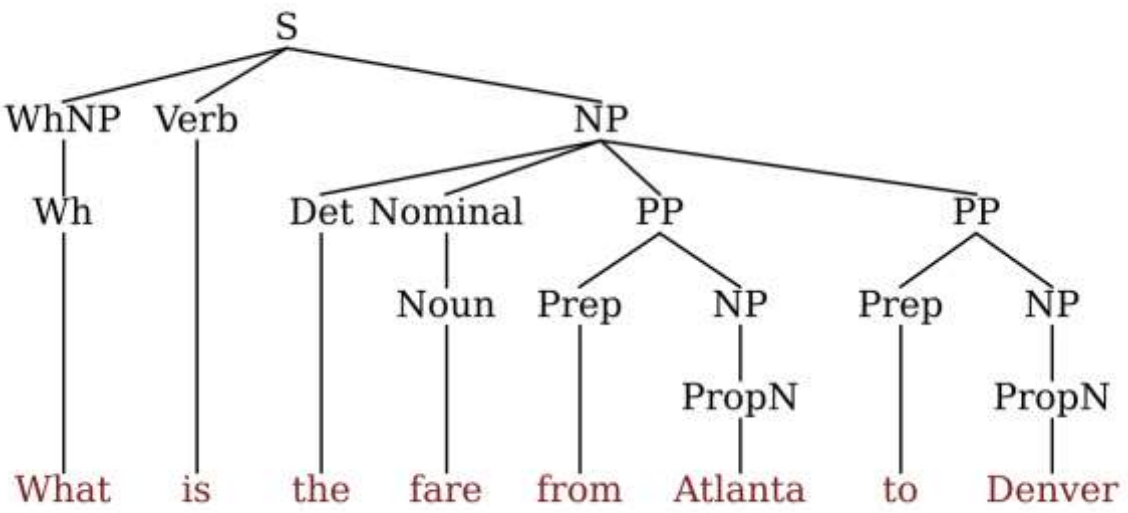
This is a wh-question with the rule S -> WhNP Verb NP. The WhNP is the single Wh word What, the Verb is is, and the NP is Det Nominal PP PP: the fare followed by two prepositional phrases, from Atlanta and to Denver.

Output:

```
python q1_atis_trees.py

Sentence (f): What is the fare from Atlanta to Denver
Number of parses: 1
(S
  (WhNP (Wh What))
  (Verb is)
  (NP
    (Det the)
    (Nominal (Noun fare))
    (PP (Prep from) (NP (PropN Atlanta)))
    (PP (Prep to) (NP (PropN Denver)))))
```

Tree structure:



g. Is there an American airlines flight from Philadelphia to Dallas?

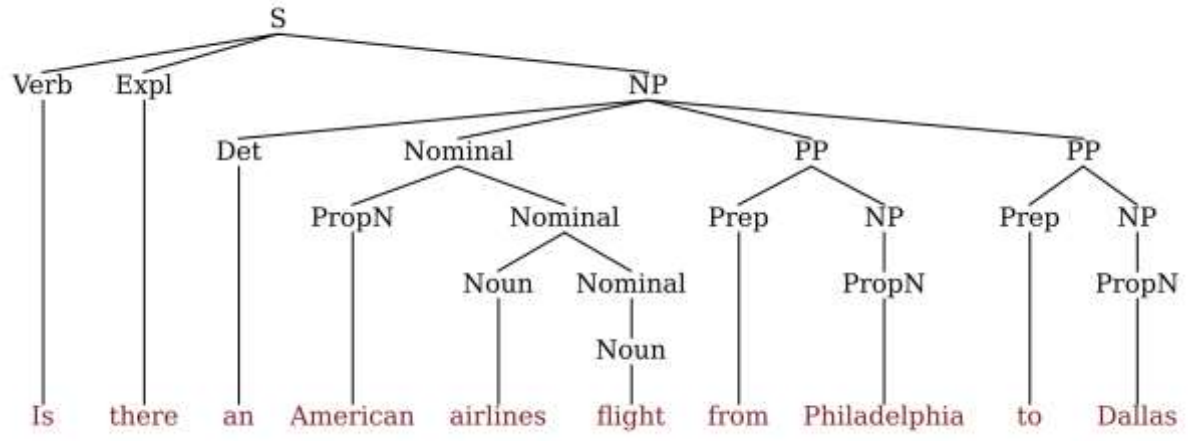
This is an existential question with the rule S -> Verb Expl NP, where Is is the Verb and there is the expletive Expl. The NP is Det Nominal PP PP: the Nominal American airlines flight is a compound of a PropN and two Nouns, followed by the prepositional phrases from Philadelphia and to Dallas.

Output:

```
python q1_atis_trees.py

Sentence (g): Is there an American airlines flight from Philadelphia to Dallas
Number of parses: 1
(S
  (Verb Is)
  (Expl there)
  (NP
    (Det an)
    (Nominal
      (PropN American)
      (Nominal (Noun airlines) (Nominal (Noun flight))))
    (PP (Prep from) (NP (PropN Philadelphia)))
    (PP (Prep to) (NP (PropN Dallas)))))
```

Tree structure:



Question 2

Consider the below grammar to construct the Early parser for the input string "Christ Opened the eyes". (10M) (CO3) (BLT4)

S -> NP VP	VP -> V NP PP	V -> opened
NP -> N	PP -> Prep NP	V -> ate
NP -> Det NP	N -> Christ	Det -> the
NP -> NP NP	N -> blind	Det -> a
VP -> V	N -> eyes	Prep -> of
VP -> V NP	N -> bread	Prep -> in

Approach

The Earley parser is a top-down, dynamic-programming chart parser. For an input of n words it builds n + 1 chart entries. Every state has the form A -> α • β [i, j], which means that the rule A -> α β has been started at position i, the part α before the dot has been recognised up to position j, and the part β after the dot is still expected. Parsing starts with the dummy state γ -> • S [0,0] in Chart[0], and each state is processed with one of three operations:

- **Predictor:** if the symbol after the dot is a non-terminal, one new state with the dot at the start is added for every rule that expands that non-terminal.
- **Scanner:** if the symbol after the dot is a part of speech (N, V, Det or Prep) and the next input word belongs to it, a completed lexical state is added to the next chart entry.
- **Completer:** when a state has its dot at the end, every earlier state that was waiting for that non-terminal is copied forward with its dot moved one symbol to the right.

The input is accepted when the state γ -> S • [0, n] appears in the last chart entry. In this program the input word Opened is matched with the lexical entry opened without regard to case.

Program: q2_earley_parser.py

```
from nltk import Tree

from tree_draw import save_tree

GRAMMAR = {
    "S": [("NP", "VP")],
    "NP": [("N",), ("Det", "NP"), ("NP", "NP")],
    "VP": [("V",), ("V", "NP"), ("V", "NP", "PP")],
    "PP": [("Prep", "NP")],
}

LEXICON = {
    "N": ["Christ", "blind", "eyes", "bread"],
    "V": ["opened", "ate"],
    "Det": ["the", "a"],
    "Prep": ["of", "in"],
}

CANONICAL = {
    word.lower(): word for words in LEXICON.values() for word in words
```

```
}


class State:
    def __init__(self, lhs, rhs, dot, start, end, action, children):
        self.lhs = lhs
        self.rhs = rhs
        self.dot = dot
        self.start = start
        self.end = end
        self.action = action
        self.children = children
        self.number = None

    def next_symbol(self):
        if self.dot < len(self.rhs):
            return self.rhs[self.dot]
        return None

    def key(self):
        return (self.lhs, self.rhs, self.dot, self.start, self.end)

    def __str__(self):
        symbols = list(self.rhs)
        symbols.insert(self.dot, "\u2022")
        body = " ".join(symbols)
        return (
            f"S{self.number:<3} {self.lhs:<4}-> {body:<14} "
            f"[{self.start},{self.end}]  {self.action}"
        )



def earley_parse(words):
    size = len(words) + 1
    chart = [[] for _ in range(size)]
    seen = [set() for _ in range(size)]
    counter = [0]

    def enqueue(state, index):
        if state.key() in seen[index]:
            return
        seen[index].add(state.key())
        state.number = counter[0]
        counter[0] += 1
        chart[index].append(state)

    def predictor(state, index):
        symbol = state.next_symbol()
        for rhs in GRAMMAR[symbol]:
            enqueue(State(symbol, rhs, 0, index, index, "Predictor", []), index)

    def scanner(state, index):
        category = state.next_symbol()
        if index < len(words) and words[index] in LEXICON[category]:
            word = words[index]
            enqueue(
                State(category, (word,), 1, index, index + 1, "Scanner", [word]),
                index + 1,
            )

    def completer(state, index):
        position = 0
        while position < len(chart[state.start]):
            waiting = chart[state.start][position]
            position += 1
            if waiting.next_symbol() == state.lhs:
                advanced = State(
                    waiting.lhs,
                    waiting.rhs,
                    waiting.dot + 1,
```

```
                    waiting.start,
                    index,
                    "Completer",
                    waiting.children + [state],
                )
                enqueue(advanced, index)

    enqueue(State("\u03b3", ("S",), 0, 0, 0, "Dummy start state", []), 0)
    for index in range(size):
        position = 0
        while position < len(chart[index]):
            state = chart[index][position]
            position += 1
            symbol = state.next_symbol()
            if symbol is None:
                completer(state, index)
            elif symbol in LEXICON:
                scanner(state, index)
            else:
                predictor(state, index)
    return chart



def build_tree(state):
    if state.lhs in LEXICON:
        return Tree(state.lhs, [state.children[0]])
    return Tree(state.lhs, [build_tree(child) for child in state.children])



def main():
    sentence = "Christ Opened the eyes"
    words = [CANONICAL.get(word.lower(), word) for word in sentence.split()]
    chart = earley_parse(words)
    print(f"Input  : {sentence}")
    print(f"Tokens : {' '.join(words)}")
    print()
    for index, states in enumerate(chart):
        if index == 0:
            print("Chart[0]  (before reading any word)")
        else:
            print(f"Chart[{index}]  (after reading '{words[index - 1]}')")
        for state in states:
            print(state)
        print()
    final = [
        state
        for state in chart[-1]
        if state.lhs == "\u03b3" and state.dot == 1 and state.start == 0
    ]
    if final:
        accepted = final[0]
        label = f"S{accepted.number}  \u03b3 -> S \u2022  [0,{len(words)}]"
        print(f"Accepting state: {label}")
        print("Result: the input string is accepted by the grammar")
        tree = build_tree(accepted.children[0])
        print()
        print(tree.pformat(margin=40))
        save_tree(tree, "tree_q2.png")
    else:
        print("Result: the input string is rejected")



if __name__ == "__main__":
    main()
```

Output

Chart[0] and Chart[1]

Chart[0] starts with the dummy state S0, and the Predictor expands S and then NP at position 0. In Chart[1] the Scanner reads Christ as N (S5). The Completer then finishes NP -> N • (S6), moves the dots in S -> NP • VP (S7) and NP -> NP • NP (S8), and the Predictor adds the VP rules and the NP rules at position 1.

```
python q2_earley_parser.py

Input  : Christ Opened the eyes
Tokens : Christ opened the eyes

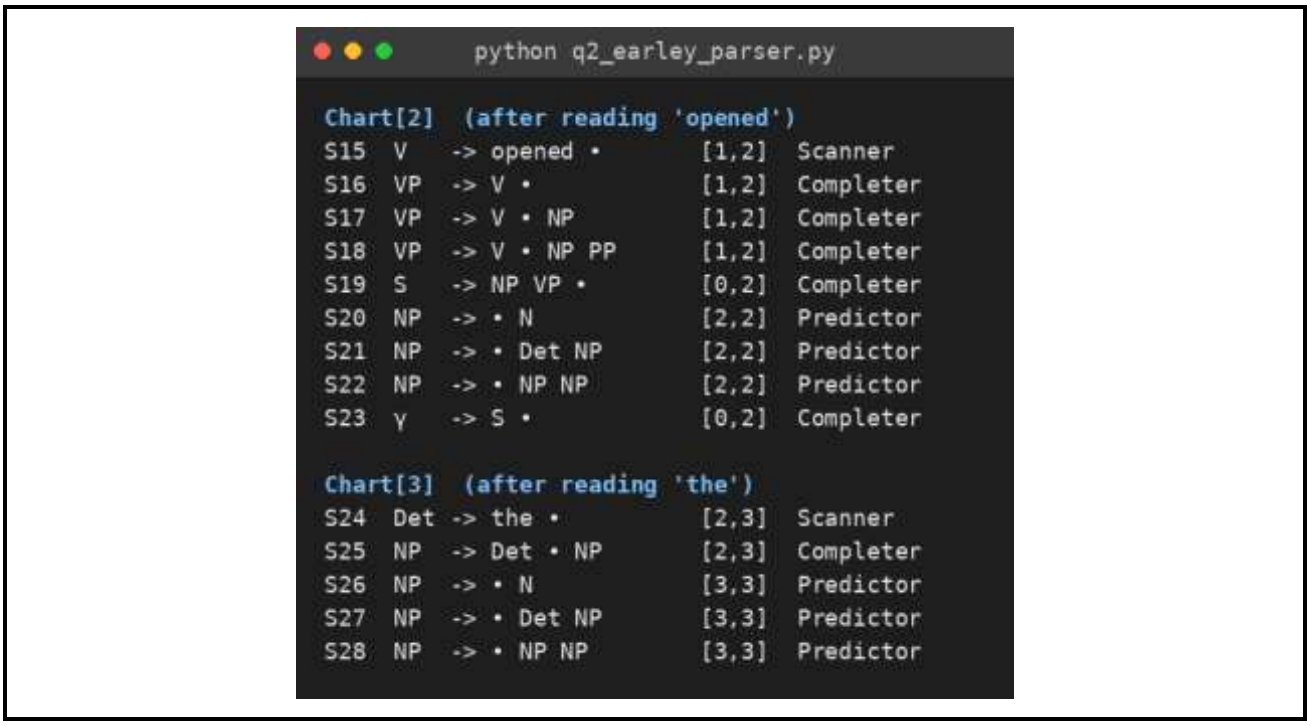
Chart[0]  (before reading any word)
S0   γ   -> • S            [0,0]  Dummy start state
S1   S   -> • NP VP        [0,0]  Predictor
S2   NP  -> • N            [0,0]  Predictor
S3   NP  -> • Det NP       [0,0]  Predictor
S4   NP  -> • NP NP        [0,0]  Predictor

Chart[1]  (after reading 'Christ')
S5   N   -> Christ •       [0,1]  Scanner
S6   NP  -> N •            [0,1]  Completer
S7   S   -> NP • VP        [0,1]  Completer
S8   NP  -> NP • NP        [0,1]  Completer
S9   VP  -> • V            [1,1]  Predictor
S10  VP  -> • V NP         [1,1]  Predictor
S11  VP  -> • V NP PP      [1,1]  Predictor
S12  NP  -> • N            [1,1]  Predictor
S13  NP  -> • Det NP       [1,1]  Predictor
S14  NP  -> • NP NP        [1,1]  Predictor
```

Chart[2] and Chart[3]

In Chart[2] the Scanner reads opened as V (S15). The Completer produces VP -> V • (S16), VP -> V • NP (S17) and VP -> V • NP PP (S18), and S -> NP VP • [0,2] (S19). The state γ -> S • [0,2] (S23) covers only two of the four words, so it is not the accepting state. The Predictor adds the NP rules at

position 2. In Chart[3] the Scanner reads the as Det (S24), the Completer gives NP -> Det • NP (S25), and the Predictor adds the NP rules at position 3.



Chart[4], acceptance and parse tree

In Chart[4] the Scanner reads eyes as N (S29). The Completer then gives NP -> N • [3,4] (S30), NP -> Det NP • [2,4] (S31), VP -> V NP • [1,4] (S33), S -> NP VP • [0,4] (S39) and finally γ -> S • [0,4]

(S41). The last state spans the whole input, so the string is accepted. The tree below is recovered from the completed states.

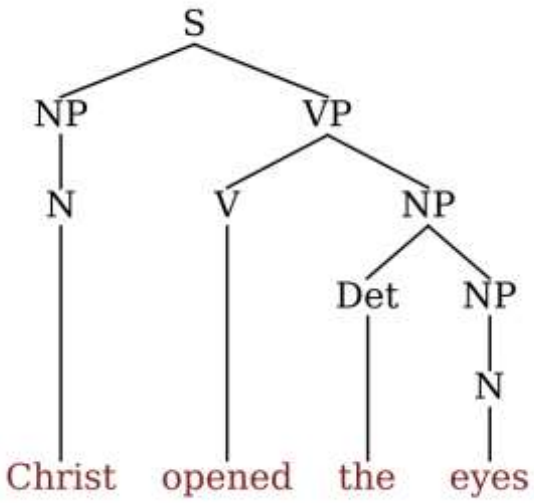
```
python q2_earley_parser.py

Chart[4]  (after reading 'eyes')
S29  N   -> eyes •          [3,4]  Scanner
S30  NP  -> N •             [3,4]  Completer
S31  NP  -> Det NP •        [2,4]  Completer
S32  NP  -> NP • NP         [3,4]  Completer
S33  VP  -> V NP •          [1,4]  Completer
S34  VP  -> V NP • PP       [1,4]  Completer
S35  NP  -> NP • NP         [2,4]  Completer
S36  NP  -> • N             [4,4]  Predictor
S37  NP  -> • Det NP        [4,4]  Predictor
S38  NP  -> • NP NP         [4,4]  Predictor
S39  S   -> NP VP •         [0,4]  Completer
S40  PP  -> • Prep NP       [4,4]  Predictor
S41  γ   -> S •             [0,4]  Completer

Accepting state: S41  γ -> S •  [0,4]
Result: the input string is accepted by the grammar

(S
  (NP (N Christ))
  (VP
    (V opened)
    (NP (Det the) (NP (N eyes)))))
```

Parse tree:



Conclusion

The Earley parser reaches the accepting state γ -> S • [0,4], so “Christ Opened the eyes” is a valid sentence of the given grammar. Its derivation is S -> NP VP, NP -> N (Christ), VP -> V NP, V -> opened, NP -> Det NP, Det -> the, NP -> N and N -> eyes.